

```
[0]: import pandas as pd
```

```
[2]: df = pd.read_csv("C:/Machine Learning/Machine Learning/Data Analysis/Notes/03-Pandas/mpg.csv")
```

```
[3]:
```

	mpg	cylinders	displacement	horsepower	weight	acceleration	model_year	origin	name
0	18.0	8	307.0	130	3504	12.0	70	1	chevrolet chevelle malibu
1	15.0	8	350.0	165	3693	11.5	70	1	buick skylark 320
2	18.0	8	318.0	150	3436	11.0	70	1	plymouth satellite
3	16.0	8	304.0	150	3433	12.0	70	1	amc rebel sst
4	17.0	8	302.0	140	3449	10.5	70	1	ford torino
...	...	...	...	...	...	...	...	...	...
393	27.0	4	140.0	86	2790	15.6	82	1	ford mustang gl
394	44.0	4	97.0	52	2130	24.6	82	2	vw pickup
395	32.0	4	135.0	84	2295	11.6	82	1	dodge rampage
396	28.0	4	120.0	79	2625	18.6	82	1	ford ranger
397	31.0	4	119.0	82	2720	19.4	82	1	chevy s-10

398 rows x 9 columns

```
[5]: df['model_year']
```

```
[5]:
```

```
0    70
1    70
2    70
3    70
4    70
...
393   82
394   82
395   82
396   82
397   82
Name: model_year, Length: 398, dtype: int64
```

```
[4]: df['model_year'].unique()
```

```
[4]:
```

```
array([70, 71, 72, 73, 74, 75, 76, 77, 78, 79, 80, 81, 82])
```

```
[6]: df['model_year'].value_counts()
```

```
[6]:
```

```
model_year
73    40
78    36
76    34
82    31
75    30
70    29
79    29
79    29
80    29
81    29
71    28
72    28
77    26
74    27
Name: count, dtype: int64
```

Let's choose model_year as categorical column for this instance.

How has miles-per-gallon changed on average per year?

In old Pandas:

```
[11]: df.groupby('model_year').mean()
```

```
Traceback (most recent call last)
TypeError
Cell In[11], line 1
----> 1 df.groupby('model_year').mean()

File c:\Users\spahali\AppData\Local\Programs\Python\Python311\lib\site-packages\pandas\core\groupby\groupby.py:226, in GroupBy.mean(self, numeric_only, skipna, engine, engine_kwargs)
    224         """
    225         """
    226         """
    227         """
    228         """
    229         """
    230         """
    231         """
    232         """
    233         """
    234         """
    235         """
    236         """
    237         """
    238         """
    239         """
    240         """
    241         """
    242         """
    243         """
    244         """
    245         """
    246         """
    247         """
    248         """
    249         """
    250         """
    251         """
    252         """
    253         """
    254         """
    255         """
    256         """
    257         """
    258         """
    259         """
    260         """
    261         """
    262         """
    263         """
    264         """
    265         """
    266         """
    267         """
    268         """
    269         """
    270         """
    271         """
    272         """
    273         """
    274         """
    275         """
    276         """
    277         """
    278         """
    279         """
    280         """
    281         """
    282         """
    283         """
    284         """
    285         """
    286         """
    287         """
    288         """
    289         """
    290         """
    291         """
    292         """
    293         """
    294         """
    295         """
    296         """
    297         """
    298         """
    299         """
    300         """
    301         """
    302         """
    303         """
    304         """
    305         """
    306         """
    307         """
    308         """
    309         """
    310         """
    311         """
    312         """
    313         """
    314         """
    315         """
    316         """
    317         """
    318         """
    319         """
    320         """
    321         """
    322         """
    323         """
    324         """
    325         """
    326         """
    327         """
    328         """
    329         """
    330         """
    331         """
    332         """
    333         """
    334         """
    335         """
    336         """
    337         """
    338         """
    339         """
    340         """
    341         """
    342         """
    343         """
    344         """
    345         """
    346         """
    347         """
    348         """
    349         """
    350         """
    351         """
    352         """
    353         """
    354         """
    355         """
    356         """
    357         """
    358         """
    359         """
    360         """
    361         """
    362         """
    363         """
    364         """
    365         """
    366         """
    367         """
    368         """
    369         """
    370         """
    371         """
    372         """
    373         """
    374         """
    375         """
    376         """
    377         """
    378         """
    379         """
    380         """
    381         """
    382         """
    383         """
    384         """
    385         """
    386         """
    387         """
    388         """
    389         """
    390         """
    391         """
    392         """
    393         """
    394         """
    395         """
    396         """
    397         """
    398         """
    399         """
    400         """
    401         """
    402         """
    403         """
    404         """
    405         """
    406         """
    407         """
    408         """
    409         """
    410         """
    411         """
    412         """
    413         """
    414         """
    415         """
    416         """
    417         """
    418         """
    419         """
    420         """
    421         """
    422         """
    423         """
    424         """
    425         """
    426         """
    427         """
    428         """
    429         """
    430         """
    431         """
    432         """
    433         """
    434         """
    435         """
    436         """
    437         """
    438         """
    439         """
    440         """
    441         """
    442         """
    443         """
    444         """
    445         """
    446         """
    447         """
    448         """
    449         """
    450         """
    451         """
    452         """
    453         """
    454         """
    455         """
    456         """
    457         """
    458         """
    459         """
    460         """
    461         """
    462         """
    463         """
    464         """
    465         """
    466         """
    467         """
    468         """
    469         """
    470         """
    471         """
    472         """
    473         """
    474         """
    475         """
    476         """
    477         """
    478         """
    479         """
    480         """
    481         """
    482         """
    483         """
    484         """
    485         """
    486         """
    487         """
    488         """
    489         """
    490         """
    491         """
    492         """
    493         """
    494         """
    495         """
    496         """
    497         """
    498         """
    499         """
    500         """
    501         """
    502         """
    503         """
    504         """
    505         """
    506         """
    507         """
    508         """
    509         """
    510         """
    511         """
    512         """
    513         """
    514         """
    515         """
    516         """
    517         """
    518         """
    519         """
    520         """
    521         """
    522         """
    523         """
    524         """
    525         """
    526         """
    527         """
    528         """
    529         """
    530         """
    531         """
    532         """
    533         """
    534         """
    535         """
    536         """
    537         """
    538         """
    539         """
    540         """
    541         """
    542         """
    543         """
    544         """
    545         """
    546
```

```
2278 grouped_mean,
```

```

2279         executor.float_dtype_mapping,
(...): 2282         skipna=skipna,

```

```

2294 self:
2295     result = self._cython_agg_general(
2296         "mean",
2297         all_lambda_x: Series(x, copy=False).mean(
2298             numeric_only=numeric_only, skipna=skipna
2299         ),
2300         numeric_only=numeric_only,
2301         skipna=skipna,
2302     )
2303     return result._finalize_(_obj=obj, method="groupby")

```

```

File ~/Users/lsbatala/Local/Programs/Python/Python314/1ib/site-packages/pandas/core/groupby/groupby.py:1800, in Groupby._cython_agg_general(self, h
ew, alt, numeric_only, min_count, **kwargs)
1800     result = self._agg.py_fallback(how, values, ndim=data.ndim, alt=alt)
1801     return result
1802     >>> new_mgr = data._grouped_reduce(array_func
1803 res = self._wrap_agg_manager(new_mgr)
1804 # how in ["idmin", "idmax"]
1805 # copy expects how to be literal["idmin", "idmax"].

```

```

File ~/Users/lsbatala/Local/Programs/Python/Python314/1ib/site-packages/pandas/core/interchange/managers.py:165, in BlockManager.grouped_reduce(sel
f, func)
1652     result_blocks = extend_blocks(result_blocks)
1653     agg =
1654     applied = blk.apply(func)
1655     result_blocks = extend_blocks(applied, result_blocks)
1656     if not (result_blocks) == 0:

```

```

File ~/Users/lsbatala/Local/Programs/Python/Python314/1ib/site-packages/pandas/core/interchange/blocks.py:56, in Block.apply(self, func, **kwargs)
541 @final
542 def apply(self, func, **kwargs) -> list [Block]:
543     """
544     apply the function to my values; return a block if we are not
545     one
546     """
547     result = func(self.values, **kwargs)
548     result = maybe_coerce_values(result)
549     _, split_op, result(result)

```

```

File ~/Users/lsbatala/Local/Programs/Python/Python314/1ib/site-packages/pandas/core/groupby/groupby.py:1789, in Groupby._cython_agg_general(clocals
array, func(values)
1781 def array_func(values: ArrayLike) -> ArrayLike:
1782     """
1783     """
1784     result = self._groupby._cython_operation(
1785         "aggregate",
1786         values=values,
1787         how=how,
1788         axis=data.ndim - 1,
1789         min_count=min_count,
1790         **kwargs,
1791     )
1792     # Not implemented for error
1793     # generally if we have numeric_only=False
1794     # and non-applicable functions
1795     # try to python-agg
1796     # TODO: shouldn't min_count matter?
1797     # TODO: avoid special casing SparseArray here
1798     # how in ["any", "all"] and non_agg (values, SparseArray):

```

```

File ~/Users/lsbatala/Local/Programs/Python/Python314/1ib/site-packages/pandas/core/groupby/ops.py:963, in BaseGroupby._cython_operation(self, kind
values, how, axis, min_count, kind, ngroups, **kwargs)
963     kind in ["transform", "aggregate"]
964     cy_op = WrappedCythonOp(kind=kind, how=how, has_dropped_na=has_dropped_na)
965     values=values
966     axis=axis,
967     min_count=min_count,
968     comp_id=self._id,
969     ngroups=ngroups,
970     **kwargs,

```

```

File ~/Users/lsbatala/Local/Programs/Python/Python314/1ib/site-packages/pandas/core/groupby/ops.py:753, in WrappedCythonOp._cython_operation(self, v
axis, axis, min_count, comp_id, ngroups, **kwargs)
753     _validate_axis(axis, values)
754     self._validate_axis(axis, values)
755     if not (values, np.ndarray):
756         # i.e. ExtensionArrays
757         values = values._groupby_opi
758         how=how
759         has_dropped_na=has_dropped_na
760         min_count=min_count,
761         ngroups=ngroups,
762         id=comp_id,
763         **kwargs,
764         self._cython_op_ndim_compat(
765             values,
766             min_count=min_count,
767             **kwargs,
768         )

```

```

File ~/Users/lsbatala/Local/Programs/Python/Python314/1ib/site-packages/pandas/core/arraylike/array.py:2080, in ExtensionArray._groupby_op(self, how, h
as_dropped_na, min_count, ngroups, id, **kwargs)
2080     def __getitem__(self, i: dtype, StringDType):
2081         # StringDType
2082         # op.how in [
2083             "prod",
2084             "mean",
2085             "kurt",
2086         ]
2087     ):
2088         raise TypeError(
2089             "ndarray TypeError(
2090                 'dtype' does not support operation 'how')")
2091     # op.how not in ["any", "all"]
2092     # fail early to avoid conversion to object
2093     _get_cython_function(op.kind, op.how, np.dtype(dtype), kind)
2094     TypeError: dtype 'str' does not support operation 'mean'

```

In new Pandas:

```

df.groupby("model_year").mean(numeric_only=True)

```

```

mpg cylinders displacement weight acceleration origin

```

72	18.714286	5.831429	218.375000	3237.714286	15.125000	1.353714
73	17.100000	6.375000	256.875000	3419.025000	14.312500	1.375000
74	22.703704	5.259259	171.740741	3077.925926	16.201704	1.666667
75	20.266667	5.600000	205.533333	3176.800000	16.050000	1.666667
76	21.573529	5.647059	197.794118	3078.735294	15.941716	1.470588
77	23.375000	5.464286	191.382857	2997.357143	14.531714	1.571429
78	24.061111	5.361111	177.805556	2861.805556	15.805556	1.611111
79	25.093103	5.827586	206.686955	3055.344828	15.811793	1.275462
80	33.696552	4.137931	115.827586	2436.655172	16.934483	2.206897
81	30.334483	4.620690	135.310345	2522.931034	16.300897	1.936517
82	31.709677	4.193548	128.870968	2453.548387	16.638710	1.645161

```
70 17.48955
71 21.250000
72 18.714286
73 17.180000
74 22.703703
75 20.266667
76 21.575229
77 23.375000
78 24.061111
79 25.693103
80 33.696552
81 30.334483
82 33.709077
Name: mpg, dtype: float64

We see that it gave only mpg values.

Let's try grouping with multiple columns.

[17]: df.groupby(['model_year', 'cylinders']).mean(numeric_only=True)

[17]:
```

		mpg	displacement	weight	acceleration	origin	
	model_year	cylinders					
		4	25.285714	107.000000	2292.571429	16.000000	2.285714
	70	6	20.500000	199.000000	2710.500000	15.500000	1.000000
		8	14.111111	367.555556	3940.555556	11.194444	1.000000
		4	27.461538	101.846154	2056.384615	16.961538	1.923077

	8	13.242871	371.714286	4537.714286	12.214286	1.000000
	3	19.000000	70.000000	2330.000000	13.500000	3.000000
72	4	23.428571	111.535714	2382.642857	17.214286	1.925713
	8	13.615385	344.846154	4228.384615	13.000000	1.000000
	3	18.000000	70.000000	2124.000000	13.500000	3.000000
	4	22.727273	109.272727	2338.090909	17.136364	2.000000
73	6	19.000000	212.250000	2917.125000	15.687500	1.250000
	8	13.200000	365.250000	4279.050000	12.250000	1.000000
	4	27.800000	96.533333	2151.466667	16.400000	2.200000
74	6	17.871143	230.428571	3320.000000	16.857143	1.000000
	8	14.200000	315.200000	4438.400000	14.700000	1.000000
	4	25.250000	114.833333	2489.250000	15.833333	2.166667
75	6	17.583333	233.750000	3398.333333	17.708333	1.000000
	8	15.666667	330.500000	4108.833333	13.166667	1.000000
	4	26.766667	106.333333	2306.600000	16.866667	1.866667
76	6	20.900000	221.400000	3349.600000	17.000000	1.300000
	8	14.666667	324.000000	4064.666667	13.222222	1.000000
	8	14.000000	284.000000	4009.000000	13.222222	1.000000

[illegible][illegible]

70	6	20.500000	199.000000	2710.500000	15.500000	1.000000
	8	14.111111	367.555556	3940.055556	11.194444	1.000000
	4	27.461538	101.846154	2056.384615	16.961538	1.923077
71	6	18.000000	243.375000	3171.875000	14.750000	1.000000

	8	13,428,571	371,714,286	4537,714,286	12,214,286	1,000,000
	3	19,000,000	70,000,000	2330,000,000	13,500,000	3,000,000
72	4	23,428,571	111,535,714	2382,642,857	17,214,286	1,928,571
	6	23,428,571	214,842,857	4330,342,857	13,000,000	1,000,000

73	6	13.015385	344.2401154	4228.394915	13.000000	1.000000
	3	18.000000	70.000000	2124.000000	13.500000	3.000000
	4	22.727273	109.272727	2338.999999	17.136364	2.000000
74	6	19.000000	212.250000	2917.125000	15.687500	1.250000
	8	13.200000	365.250000	4279.050000	12.250000	1.000000
	4	27.800000	96.533333	2151.466667	16.400000	2.200000
75	6	17.857143	230.428571	3320.000000	16.857143	1.000000
	8	14.200000	315.200000	4438.400000	14.700000	1.000000
	4	25.250000	114.833333	2489.250000	15.833333	2.166667
76	6	17.583333	233.750000	3398.333333	17.708333	1.000000
	8	15.666667	330.500000	4108.833333	13.166667	1.000000
	4	26.766667	106.333333	2306.600000	16.866667	1.866667
76	6	20.000000	221.400000	3349.600000	17.000000	1.300000
	4	16.666667	324.000000	4064.666667	13.222222	1.000000
	3	21.500000	80.000000	2720.000000	13.500000	3.000000

77	4	29.107143	106.500000	2205.071429	16.064286	1.857143
	6	19.500000	220.400000	3383.000000	16.900000	1.400000
	8	16.000000	335.750000	4177.500000	13.662500	1.000000
	4	29.576471	112.117647	2286.764706	16.282253	2.117647

78	6	19.066667	213.250000	3314.166667	16.391667	1.166667
	8	19.050000	300.833333	3563.333333	13.266667	1.000000
	4	31.525000	113.583333	2357.583333	15.991667	1.583333
	5	25.400000	183.000000	3530.000000	20.100000	2.000000
79	6	22.950000	205.666667	3025.833333	15.413333	1.000000
	8	18.630000	321.400000	3862.900000	15.400000	1.000000
	3	23.700000	70.000000	2420.000000	12.500000	3.000000
	4	34.612000	111.000000	2360.080000	17.144000	2.200000
80	5	36.400000	121.000000	2950.000000	19.900000	2.000000
	6	25.900000	196.500000	3145.500000	15.050000	2.000000
	4	32.814286	108.857143	2275.476190	16.466667	2.095238
	6	23.842857	184.000000	3093.571429	15.442857	1.714286
81	8	26.600000	350.000000	3725.000000	19.000000	1.000000
	4	32.071429	116.571429	2402.321429	16.703571	1.714286
82	5	30.237323	136.489796	2076.455556	16.032323	1.000000

	mpg	displacement	weight	acceleration	origin
cylinders					
4	25.285714	107.000000	2292.571429	16.000000	2.285714
6	20.500000	199.000000	2710.500000	15.500000	1.000000

```
26]:
```

	mpg	displacement	weight	acceleration	origin
model_year	cylinders				
4	25.285714	107.000000	2292.571429	16.000000	2.285714

7D	6	20.500000	199.000000	2710.500000	15.500000	1.000000
	8	14.111111	367.555556	3940.055556	11.194444	1.000000
82	4	32.071429	118.571429	2402.321429	16.703571	1.714286
	6	28.333333	225.000000	2931.666667	16.033333	1.000000

```
27) [year_cyl.loc[:,0]]
```

```
27) mpg      14.111111  
   displacement  367.555556  
   weight      3900.888889  
   acceleration  11.184444  
   accel_in    1.000000
```

```
Name: (78, 8), dtype: float64
```